JETIR.ORG ISSN: 2349-5162 | ESTD Year : 2014 | Monthly Issue
JOURNAL OF EMERGING TECHNOLOGIES AND INNOVATIVE RESEARCH (JETIR)
An International Scholarly Open Access, Peer-reviewed, Refereed Journal

CODEASSISTA

AN AI-POWERED WEB PLATFORM FOR CODE GENERATION, DEBUGGING AND TRANSLATION

[1]K.Arun Babu , [2]R.Tarun, [3]M.Ranjith Kumar , [4]P.Gowthami, [5]Sk.Khaja Khayuam

[1]Assistant Professor, [2]Student, [3]Student, [4]Student, [5]Student
[1]Computer Science and Engineering,
[1]Bapatla Engineering College, Bapatla, India

Abstract : CodeAssista is a full-stack, AI-powered web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) and integrated with the Google Gemini 2.5 Flash generative AI model. The platform provides three core capabilities to students and developers: natural-language code generation, automated code debugging, and cross-language code translation. A JWT-based authentication layer secures all AI endpoints, ensuring that only registered users can access the generation features. Mode-specific prompt engineering guides the Gemini model toward consistent, task-appropriate responses for each of the three operational modes. Experimental evaluation confirms reliable code generation across diverse prompts, accurate error identification and correction in pasted code, and successful translation across five target languages — Python, Java, JavaScript, C, and C++. The system demonstrates how modern generative AI can be cleanly embedded into a conventional web architecture using standard REST API patterns, without requiring specialized infrastructure or local model deployment.

IndexTerms - MERN Stack, Generative AI, Google Gemini, Code Generation, Code Debugging, Code Translation, JWT Authentication, React.js, Node.js, Express.js, MongoDB.

I. INTRODUCTION

The rapid advancement of large language models (LLMs) has created transformative opportunities across software engineering. Developers at every skill level now benefit from AI tools that can generate code from natural-language descriptions, automatically detect and correct bugs, and translate logic between programming languages. Despite the proliferation of standalone AI tools such as GitHub Copilot and ChatGPT, a persistent need exists for a dedicated, secure, web-based platform that integrates these capabilities into a cohesive workflow accessible without local installation.

CodeAssista addresses this gap by providing a purpose-built, cloud-compatible web application built entirely on open web standards. By combining the MERN stack with Google's Gemini 2.5 Flash model, the platform delivers high-quality AI assistance within a familiar browser interface, protected by industry-standard JWT authentication. Users register once, authenticate with email or username, and then access three distinct AI modes — Code Generation, Code Debugging, and Code Conversion — through a clean, responsive React.js front end.

This paper presents the design, implementation, and evaluation of CodeAssista. Section II reviews related work. Section III describes the system architecture and design. Section IV covers the implementation details. Section V presents results and testing. Section VI concludes and outlines future directions.

II. RELATED WORK

Chen et al. [1] demonstrated that large language models trained on publicly available code repositories can solve competitive programming problems and generate functionally correct implementations from docstring descriptions, establishing the theoretical basis for AI-assisted code generation tools. This work underpins commercial tools such as GitHub Copilot, which is powered by OpenAI Codex and integrated directly into VS Code and JetBrains IDEs.

Tabnine [2] and Amazon CodeWhisperer [3] offer AI-powered code completion within IDE environments, focusing on auto-completion rather than full code generation, debugging, or cross-language translation. These tools require IDE plugin installation and, in most cases, paid subscriptions, creating barriers for casual users and students.

Pearce et al. [4] investigated the security properties of AI-generated code and found that while LLMs produce syntactically correct code at high rates, logical correctness and security properties vary significantly with prompt formulation. This motivates the structured, mode-specific prompt engineering employed in CodeAssista to improve output consistency and reliability.

Jones et al. [5] standardized JSON Web Tokens (RFC 7519) as a compact, stateless mechanism for transmitting user identity claims between parties. This standard underpins the authentication architecture of CodeAssista, where a signed JWT is returned on login and validated on every protected API request without server-side session storage.

None of the reviewed tools provide a standalone, browser-based platform offering all three capabilities — generation, debugging, and translation — behind a proper authentication layer without IDE dependency. CodeAssista fills this gap.

III. System Architecture And Design

A. Overall Architecture

CodeAssista follows a three-tier web architecture: a presentation tier (React.js SPA), a logic tier (Node.js/Express.js REST API), and a data tier (MongoDB). An external service tier (Google Gemini API) is accessed exclusively from the logic tier, ensuring that API credentials are never exposed to the client. Fig. 1 illustrates the overall system architecture.

Fig. 1. CodeAssista System Architecture

Tier	Technology	Responsibility
Presentation	React.js 19, React Router, Axios	User interface, routing, HTTP calls
Logic	Node.js, Express.js 5, JWT, Gemini SDK	REST API, auth middleware, AI prompting
Data	MongoDB, Mongoose ODM	User document persistence
External AI	Google Gemini 2.5 Flash API	Code generation, debugging, translation

B. Database Design

MongoDB stores user data in a single collection. The Mongoose schema enforces data types and uniqueness constraints. Passwords are hashed using bcryptjs with a salt round of 10 before persistence. The schema does not store JWT tokens; authentication is entirely stateless.

Table I. User Collection Schema

Field	Type	Constraints
_id	ObjectId	Auto-generated primary key
fullName	String	Required
username	String	Required, Unique
email	String	Required, Unique
password	String	Required, bcrypt-hashed

C. REST API Design

The backend exposes a RESTful API over HTTP. All AI endpoints are guarded by JWT middleware that rejects unauthenticated requests with HTTP 401.

Table II. API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/auth/signup	None	Register new user
POST	/api/auth/login	None	Authenticate; return JWT
POST	/api/ai/generate	JWT	Submit prompt; return AI output

D. AI Prompt Engineering

Each operational mode employs a distinct, structured prompt template. In generate mode, the model is instructed to produce clean, correct, well-structured code from the user's natural-language description. In debug mode, a four-step instruction sequence directs the model to analyze the code, identify all errors, explain each clearly, and provide a corrected version. In convert mode, the model is instructed to translate the code into the selected target language while preserving the original logic and returning only the converted code.

Table III. AI Mode Prompt Summary

Mode	System Role	Key Output
generate	Expert developer — generate clean correct code	Working code matching the description
debug	Expert developer — analyze, identify, explain,	Error list + corrected code

Mode	System Role	Key Output
	correct	
convert	Professional engineer — convert to <lang>, maintain logic	Equivalent code in target language

E. Security Design

Security is addressed at multiple layers. Passwords are hashed using bcryptjs (work factor 10), making brute-force attacks computationally infeasible. JWTs are signed with HMAC-SHA256 using a server-side secret stored in the .env file and set to expire after one hour. The authMiddleware intercepts every protected request, validates the signature and expiry, and attaches the decoded userId to the request object before passing control to the route handler. Sensitive credentials (MONGO_URI, JWT_SECRET, GEMINI_API_KEY) are loaded from environment variables at startup and excluded from version control.

IV. Implementation

A. Project Structure

The codebase is divided into two top-level directories — backend and frontend — maintaining a strict separation of concerns. The backend contains the Express server entry point, Mongoose model, JWT middleware, and two route modules. The frontend contains the React SPA with four page components, a centralized Axios instance, and co-located CSS files.

B. Backend Implementation

1) Server Entry Point (server.js)

The server.js file loads environment variables first via dotenv.config(), registers CORS and JSON body-parsing middleware, mounts the auth and AI route handlers, establishes the MongoDB connection through Mongoose, and starts the HTTP listener on the configured port.

2) Authentication Middleware (authMiddleware.js)

The middleware extracts the Bearer token from the Authorization header, verifies the signature against JWT_SECRET, and attaches the decoded userId to req. Requests with missing or invalid tokens receive a 401 Unauthorized response and never reach the route handler. The implementation is shown below.

```
const authMiddleware = (req, res, next) => {
    const token = req.headers.authorization?.split(' ')[1];
          if (!token) return res.status(401).json({ message: 'No token' });
          try {
          const decoded = jwt.verify(token, process.env.JWT_SECRET);
          req.userId = decoded.userId;  next();
          } catch { return res.status(401).json({ message: 'Invalid token' }); }
          };
```

3) Authentication Routes (authRoutes.js)

The signup route validates required fields, checks for duplicate email or username using a MongoDB $or query, hashes the password with bcrypt, and persists the document. The login route accepts either email or username as the identifier, verifies the bcrypt hash, and returns a signed JWT alongside the user's public profile data.

4) AI Routes (aiRoutes.js)

The AI route handler receives prompt, mode, and optional targetLanguage from the request body. Based on mode, it constructs the appropriate prompt string, calls model.generateContent() via the Google Generative AI SDK, and returns the text response as JSON.

```
const model = genAI.getGenerativeModel({ model: 'models/gemini-2.5-flash' });
const result = await model.generateContent(finalPrompt);
res.json({ output: result.response.text() });
```

C. Frontend Implementation

1) Route Configuration (App.js)

App.js defines four client-side routes using React Router v7. The /app route is protected: if no JWT is found in localStorage, the user is redirected to /login via the Navigate component. The token state is initialized from localStorage and updated on successful login.

2) Code Assistant Component (CodeAssistant.js)

This is the core of the application. Five state variables manage the interaction: mode, prompt, result, loading, and targetLanguage. The mode-switch toolbar resets input and output on each switch. The textarea placeholder adapts to the current mode. The language dropdown renders exclusively in convert mode. The generateCode function validates the token and prompt, posts to /api/ai/generate with the Bearer token, and displays the AI response in a scrollable monospace output panel.

Table IV. Frontend Component Summary

Component	Route	Key Responsibilities
Home.js	/	Landing page: hero, features, tech stack, footer
Login.js	/login	Split-screen auth form; stores JWT in localStorage
Signup.js	/signup	Registration form; redirects to /login on success
CodeAssistant.js	/app	Mode switch, prompt input, AI call, output render

D. Technology Stack Summary

Table V. Full Technology Stack

Layer	Package	Version	Purpose
Frontend	react	^19.2.4	Core UI library
Frontend	react-router-dom	^7.13.0	Client-side SPA routing
Frontend	axios	^1.13.4	HTTP client
Frontend	react-markdown	^10.1.0	Markdown rendering for AI output
Backend	express	^5.2.1	REST API framework
Backend	mongoose	^9.1.5	MongoDB ODM
Backend	bcryptjs	^3.0.3	Password hashing
Backend	jsonwebtoken	^9.0.3	JWT generation and verification
Backend	@google/generative-ai	^0.24.1	Gemini AI SDK
Backend	dotenv	^17.2.3	Environment variable management

V. RESULTS AND DISCUSSION

A. Authentication Testing

All authentication scenarios were tested systematically. Signup correctly rejects duplicate email/username with HTTP 400, rejects incomplete requests with 'All fields are required', and returns HTTP 201 on valid unique registration. Login correctly accepts both email and username as identifiers, returns HTTP 400 with 'Invalid credentials' for wrong passwords or non-existent accounts, and returns a signed JWT on success. The authMiddleware correctly returns 401 for missing, malformed, and expired tokens.

B. Code Generation Testing

Code generation was evaluated across 20 diverse prompts spanning Python, JavaScript, Java, SQL, and React. Representative test cases are shown in Table VI.

Table VI. Code Generation Test Cases

#	Prompt Description	Language	Result
1	Recursive factorial function	Python	PASS
2	Debounce utility for event handlers	JavaScript	PASS
3	Binary search implementation	Java	PASS
4	Second-highest salary SQL query	SQL	PASS
5	Responsive navigation bar component	React JSX	PASS
6	Linked list insert and delete	C++	PASS
7	REST API endpoint for user profile	Node.js/Express	PASS
8	Bubble sort algorithm	C	PASS

C. Code Debugging Testing

Debugging was evaluated on 10 code samples containing intentional errors including off-by-one loop bounds, missing await keywords, uninitialized pointer dereferences, null reference omissions, and type mismatches. In all 10 cases the model correctly identified the error(s), provided a clear explanation, and returned a corrected version of the code.

D. Code Conversion Testing

Conversion was tested across 15 source-target language pairs. Table VII shows a representative subset.

Table VII. Code Conversion Test Cases

#	Source Language	Target Language	Logic Preserved	Syntax Correct	Result
1	Python	Java	Yes	Yes	PASS
2	JavaScript	Python	Yes	Yes	PASS
3	C++	Python	Yes	Yes	PASS
4	Java	C	Yes	Yes	PASS
5	Python	JavaScript	Yes	Yes	PASS

E. Performance Metrics

Table VIII. System Performance Metrics

Metric	Observed Value	Notes
AI response latency	2.1 – 4.8 seconds	Dependent on prompt length and API load
Login response time	< 200 ms	Includes bcrypt verify + JWT sign
Signup response time	< 250 ms	Includes bcrypt hash + MongoDB write
JWT verification overhead	< 5 ms	HMAC-SHA256 synchronous
Frontend initial load time	< 1.5 seconds	React SPA, CRA build

F. Comparative Analysis

Table IX compares CodeAssista with the most prominent existing AI code assistance tools across five dimensions relevant to students and developers.

Table IX. Comparison with Existing Tools

Feature	CodeAssista	GitHub Copilot	ChatGPT	Tabnine
Code Generation	Yes	Yes	Yes	Partial
Code Debugging	Yes	Partial	Yes	No
Code Translation	Yes	No	Yes	No
User Authentication	Yes (JWT)	GitHub OAuth	OpenAI Account	Account
Browser-Based (no IDE)	Yes	No	Yes	No
Free / Open Source	Yes	Paid	Limited Free	Limited Free
Structured Mode UI	Yes	No	No	No

The comparison confirms that CodeAssista is the only tool in the evaluated set that combines all three code assistance modes (generation, debugging, translation) with a dedicated structured UI, browser-based access without IDE dependency, and a proper JWT authentication layer, all freely available.

V. Conclusion

This paper presented CodeAssista, a full-stack MERN web application that embeds Google Gemini 2.5 Flash generative AI to deliver three practical code assistance capabilities: natural-language code generation, automated debugging, and cross-language translation. The system is secured by JWT-based authentication and employs structured, mode-specific prompt engineering to ensure consistent AI output quality.

Testing across all functional areas confirms correct authentication behavior, reliable AI-assisted code generation, accurate error identification and correction, and successful code translation across five target languages. Performance measurements demonstrate acceptable latency for both authentication operations and AI responses. The comparative analysis confirms that CodeAssista occupies a unique position among existing tools, combining all three modes with browser-based access and a proper authentication layer.

The project validates that modern generative AI can be effectively integrated into conventional MERN web architectures using standard REST API patterns, without specialized infrastructure. Future work will focus on syntax highlighting in the

output panel, multi-turn conversation history, sandboxed code execution, multi-file project support, and role-based access control for usage metering.

VI. ACKNOWLEDGMENT

The authors sincerely thank their project guide and the faculty of the Department of Computer Science and Engineering, Bapatla Engineering College, for their invaluable guidance and support throughout this work. The authors also acknowledge Google AI Studio for providing access to the Gemini API for academic purposes.

VII. REFERENCES

[1] M. Chen, J. Tworek, H. Jun et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[2] Tabnine, "Tabnine AI Code Completion," [Online]. Available: https://www.tabnine.com. [Accessed: March 2025].

[3] Amazon Web Services, "Amazon CodeWhisperer," AWS Documentation, 2023. [Online]. Available: https://docs.aws.amazon.com/codewhisperer.

[4] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in IEEE Symposium on Security and Privacy (SP), 2022, pp. 754-768.

[5] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[6] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," Technical Report, arXiv:2312.11805, 2024.

[7] A. Banks and E. Porcello, Learning React: Modern Patterns for Developing React Apps, 2nd ed. Sebastopol, CA: O'Reilly Media, 2020.

[8] E. Brown, Web Development with Node and Express, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.

[9] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA: O'Reilly Media, 2019.

[10] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in USENIX Annual Technical Conference, FREENIX Track, 1999, pp. 81-91.

[11] GitHub, Inc., "GitHub Copilot Documentation," 2024. [Online]. Available: https://docs.github.com/en/copilot.

[12] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000